Proyecto 2: Juego de reflejos en Pochoco SoC

ICC3302 Arquitectura de Computadores

Constanza Bustamante, Elena Durán, Mario Messen

Octubre 2026

1. Introducción

En este proyecto hicimos un juego de reflejos que corre en el Espino Core (Pochoco SoC) sobre la FPGA de la Go Board. Al empezar cada ronda se prenden los 4 LEDs por 3 segundos, después queda uno solo y el jugador tiene que apretar su botón lo más rápido posible. El tiempo se muestra en décimas de segundo y después de 10 aciertos se muestra el promedio. Si se aprieta un botón equivocado aparece “EE” y se repite la ronda.

Para esto modificamos el RTL del periférico, escribimos el juego en assembly (`sw/game.s`) y programamos un assembler propio en Python (`tools/assembler.py`). El juego lo probamos en la placa y funciona.

2. Arquitectura

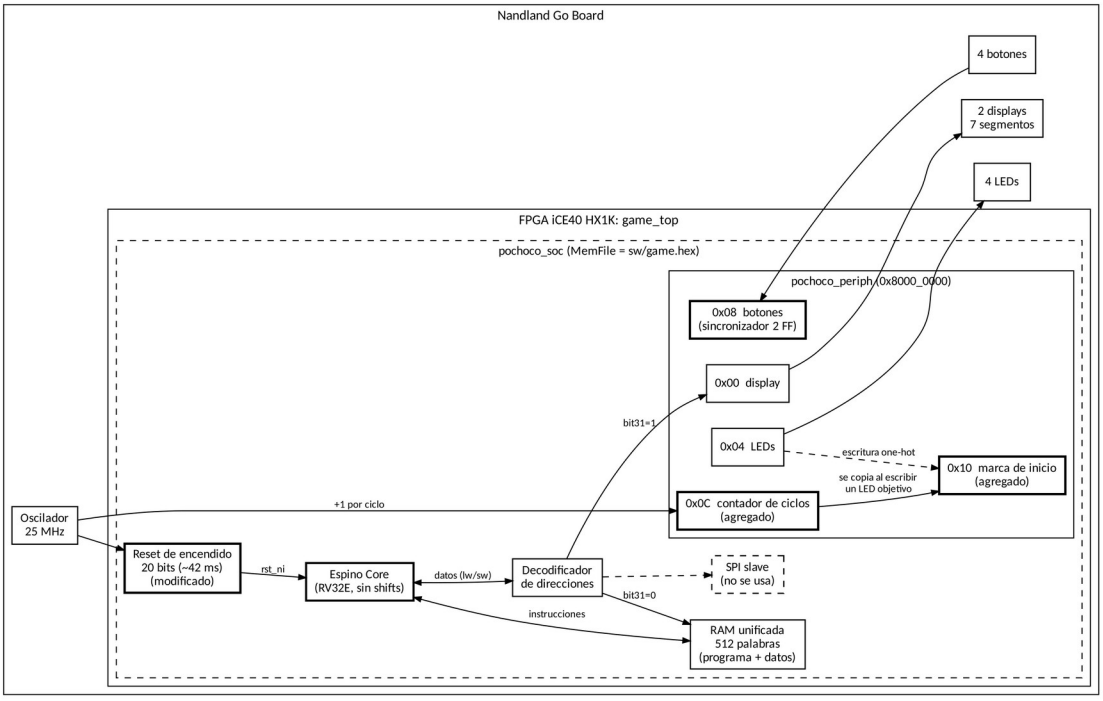


Diagrama de arquitectura del sistema. En negrita lo que agregamos o modificamos.

El Espino Core implementa RV32E (16 registros) sin multiplicación ni división, y además tiene los shifts deshabilitados. Por eso en el programa las multiplicaciones por 2 se hacen sumando un registro consigo mismo y las divisiones se hacen con restas.

`game_top.v` solo instancia el `pochoco_soc` con `MemFile = "sw/game.hex"` y conecta los pines de la placa. El programa se comunica con la placa leyendo y escribiendo en estas direcciones:

Dirección	Registro
0x8000_0000	display (escritura)
0x8000_0004	LEDs (escritura)
0x8000_0008	botones (lectura)
0x8000_000C	contador de ciclos (lectura, agregado)
0x8000_0010	marca de inicio del LED objetivo (lectura, agregado)

Cambios al RTL:

- **Contador de ciclos:** en `pochoco_periph.v` agregamos un registro de 32 bits que suma 1 en cada ciclo de reloj. Para medir un tiempo el programa lo lee dos veces y resta. Aunque el contador se desborda cada 171 s, la resta sigue dando bien porque es en módulo 2^32.
- **Marca de inicio:** cuando el programa escribe en los LEDs un valor con un solo LED prendido, el hardware copia el contador a este registro en ese mismo ciclo. Así la medición parte justo cuando se prende el LED objetivo.
- **Botones:** pasan por dos flip-flops para sincronizarlos con el reloj.
- **Reset de encendido:** en `pochoco_soc.v` lo alargamos de 15 ciclos a 2^20 ciclos (~42 ms). Sin esto el juego no partía en la placa (ver sección 6).

3. Temporización

La Go Board tiene un reloj de **25 MHz**, o sea 25.000.000 de ciclos por segundo. Con eso sacamos las constantes:

Tiempo	Ciclos
3 s (fase inicial)	75.000.000
0,1 s (una décima)	2.500.000
0,05 s (media décima, para redondear)	1.250.000

Como RV32E no puede cargar un número de 32 bits de una vez, se usan dos instrucciones. Por ejemplo 75.000.000 = 0x047868C0:

```
lui  x6, 0x4787       # parte alta
addi x6, x6, -1856    # parte baja
```

La parte baja es 0x8C0, que con signo vale −1856. Por eso la parte alta lleva +1 (0x4786 + 1).

Espera de 3 s. Se guarda el valor del contador y se repite un loop hasta que la diferencia llegue a 75.000.000:

```
    lw   x11, 0(x4)          # contador al inicio
prep_wait:
    lw   x12, 0(x4)          # contador actual
```

```
sub  x13, x12, x11       # ciclos que han pasado
bltu x13, x6, prep_wait  # si no llega a 3 s, seguir
```

Si alguien tiene un botón apretado al final de los 3 s, se espera a que lo suelte antes de mostrar el LED objetivo, así que la fase dura al menos 3 s.

Conversión de ciclos a décimas. Una décima son 2.500.000 ciclos. Como no hay división, el programa va contando las décimas mientras espera el botón: cada vez que pasan 2.500.000 ciclos suma 1. Cuando se aprieta el botón correcto, si sobran 1.250.000 ciclos o más se suma 1 más para redondear. Por ejemplo, 9.300.000 ciclos son 3 décimas y sobran 1.800.000, así que se muestra “04” (0,4 s).

4. Programa en assembly

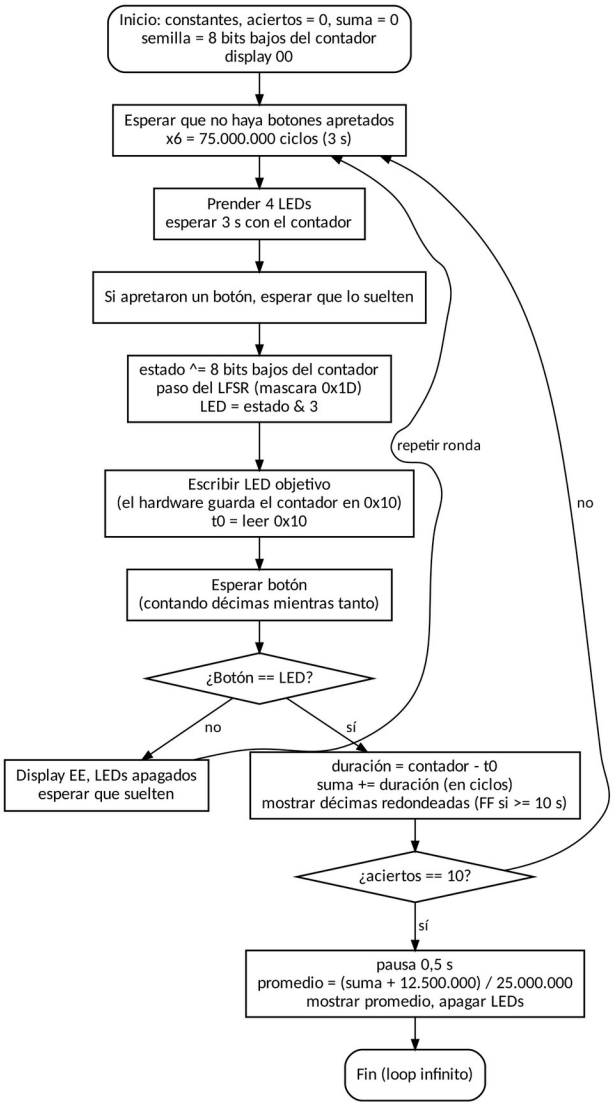


Diagrama de flujo del juego.

Registros principales: x1–x4 guardan las direcciones de display, LEDs, botones y contador; x5 es una décima en ciclos; x6 es el umbral de espera (3 s o 0,05 s); x7 cuenta los aciertos; x8 suma los tiempos en ciclos; x9 es el estado del generador aleatorio y x14 la máscara del LED objetivo.

Ronda. Se espera que no haya botones apretados, se prenden los 4 LEDs por 3 s, se elige el LED y se escribe en los LEDs. Después se leen los botones en un loop hasta que haya uno apretado:

```
    lw   x12, 0(x3)             # leer botones
    beq  x12, x0, response_wait  # ninguno: seguir esperando
    bne  x12, x14, wrong_answer  # no es el LED objetivo: error
```

Si es correcto, se calcula el tiempo, se suma a x8, se muestra en el display y se suma 1 a x7. Si es incorrecto, se muestra “EE”, se espera que suelten el botón y se repite la ronda. En ese caso x7 y x8 no cambian, así que no se pierden los aciertos.

LED pseudoaleatorio. Usamos un LFSR de 8 bits. La semilla son los 8 bits bajos del contador al encender. En cada ronda se hace XOR con los 8 bits bajos del contador (que dependen de cuánto se demoró el jugador), se desplaza el estado un bit (sumándolo consigo mismo) y si el bit que sale era 1 se hace XOR con 0x1D. El LED es `estado & 3`. El primer LED después de encender es siempre el mismo, pero los siguientes dependen de los tiempos del jugador, así que la secuencia no se repite.

Display. El display muestra cada nibble del byte como un dígito hexadecimal, así que para mostrar “37” hay que escribir 0x37. Se calculan las decenas restando 10, se multiplican por 16 sumando cuatro veces y se suman las unidades. Si el tiempo es 10 s o más se muestra “FF”.

Promedio. En x8 se suman los tiempos exactos en ciclos (no los redondeados). Al final:

```
promedio en décimas = (suma + 12.500.000) / 25.000.000
```

25.000.000 son 10 rondas × 2.500.000, y el 12.500.000 es para redondear. La división se hace con restas. Por ejemplo, si la suma es 92.000.000 el resultado es 4, y se muestra “04”.

5. Assembler

El assembler se ejecuta con `python tools\assembler.py sw\game.s -o sw\game.hex` y genera una palabra de 32 bits en hexadecimal por línea.

Sintaxis: una instrucción por línea, operandos separados por coma, comentarios con `#` o `;`, etiquetas con `nombre:`, números en decimal, hexadecimal o binario, y registros x0 a x15. Soporta las instrucciones que usa el juego: `add sub and xor addi andi xori lw sw beq bne bltu lui jal`.

Etiquetas: el assembler hace dos pasadas. En la primera calcula la dirección de cada instrucción y guarda las etiquetas. En la segunda codifica, y para los saltos calcula el desplazamiento como dirección de la etiqueta menos dirección de la instrucción.

Validaciones: da error si la instrucción no existe (incluidos los shifts), si el registro no es de x0 a x15, si faltan operandos, si un inmediato no cabe (12 bits para I/S, 13 para saltos, 21 para `jal`) o si una etiqueta está repetida o no existe.

Ejemplo de traducción: `bltu x13, x6, prep_wait` en la dirección 0x58, con `prep_wait` en 0x50.

- Desplazamiento = 0x50 − 0x58 = −8 = `1 1111 1111 1000` en 13 bits.
- Formato B: opcode 1100011, funct3 de `bltu` = 110, rs1 = x13 = 01101, rs2 = x6 = 00110.

```
imm[12] imm[10:5] rs2   rs1   funct3 imm[4:1] imm[11] opcode
1       111111    00110 01101 110    1100     1       1100011
```

Esto da `1111 1110 0110 0110 1110 1100 1110 0011` = **0xFE66ECE3**, que es lo que aparece en `game.hex`.

Pruebas: en `tests/test_assembler.py` hay 14 pruebas (`python -m unittest discover -s tests`) que revisan codificaciones conocidas, etiquetas hacia adelante y atrás, límites de registros e inmediatos, rechazo de shifts, que el juego quepa en la RAM y el cálculo del promedio. Todas pasan.

6. Problemas que tuvimos

La fase de 3 s duraba 0,05 s desde la segunda ronda. x6 se cargaba con 3 s solo al inicio, pero dentro de la ronda se reutilizaba para guardar 0,05 s. Lo arreglamos cargando x6 = 3 s al comienzo de cada ronda.

En la placa el display quedaba fijo en "00". En simulación funcionaba, así que probamos con un diseño sin procesador (`test_placa`) que sí funcionó. El problema era que las memorias BRAM de la iCE40 tardan un poco en estar listas después de programar la FPGA, y el reset de 15 ciclos dejaba partir al procesador antes. Leía ceros como instrucciones y se saltaba las primeras (como `lui x1, 0x80000`). Alargando el reset a 2^20 ciclos el juego funcionó.

La síntesis fallaba en Windows. El yosys de Windows se cae en el paso `abc9`, así que en `build.bat` usamos `abc` en su lugar.

7. Compilar y cambiar una constante

En Windows, desde la terminal de OSS CAD Suite: `build.bat` ensambla el juego, sintetiza y programa la placa.

Para cambiar la fase inicial de 3 s a 2 s (50.000.000 = 0x2FAF080) se cambian las líneas que cargan x6 al inicio de la ronda en `sw/game.s`:

```
lui  x6, 0x2faf
addi x6, x6, 128
```

y se vuelve a ejecutar `build.bat`.